
Primal Generation, Dual Judgment: Self-Training from Test-Time Scaling

Anonymous Author(s)

Affiliation

Address

email

Abstract

Code generation is typically trained in the *primal* space of programs: a model produces a candidate solution and receives sparse execution feedback, often a single pass/fail bit. Test-time scaling enriches the inference procedure by sampling multiple candidates and judging among them, but the comparative information this process reveals is discarded after inference. We argue that this information defines a *dual* judgment space that provides a far richer training signal: the model learns not from an isolated success or failure, but from the relative correctness structure across its own plausible attempts, identifying which succeed, which fail, and what distinguishes them. We introduce **DuST** (**D**ual **S**elf-**T**raining), a framework for self-training from the dual judgment space. DuST samples candidate programs from the model’s own distribution, labels them through sandbox execution, retains groups containing both successes and failures, and trains the model to rank candidates by execution correctness using GRPO. The objective is purely discriminative: the model is never directly rewarded for generating correct programs. Dual self-training improves both judgment and generation. Across four models spanning two families and three scales (4B to 30B), DuST consistently improves Best-of-4 test-time scaling on LiveCodeBench. For Qwen3-30B-Thinking on LiveCodeBench v6, judgment quality improves by +6.2 NDCG, single-sample pass@1 improves by +3.1, and Best-of-4 accuracy improves by +4.1. The trained model’s single rollout matches the base model’s Best-of-4 performance. SFT on the same ranking data improves judgment without improving generation, confirming that on-policy RL is the mechanism that transfers dual-space learning back into primal generation.

1 Introduction

Code generation is often supervised through execution: a model produces candidate code, the code is run against tests, and the resulting feedback indicates whether it is correct. This gives code generation an unusually objective learning signal, but in the standard generation setting, the signal is also highly compressed. A pass/fail outcome tells the model whether one sampled code attempt succeeded, while revealing little about how that attempt compares with other plausible candidates the model could have produced. Test-time scaling enriches this picture by sampling multiple candidate codes and judging among them [Snell et al., 2025, Yuan et al., 2024, Zhang et al., 2025, Li et al., 2025a, Zhou et al., 2025, Lee et al., Wu et al., 2025a, Muennighoff et al., 2025, Wu et al., 2025b]. In code generation, a common and effective instantiation is the “generate-then-judge” recipe: the model samples diverse candidate codes, and a judge ranks them to identify the most promising one [Ma et al., 2025, Yu et al., 2025b, Aggarwal et al., 2025, Li et al., 2025b, Wang et al.]. This paradigm is especially useful in realistic coding settings where hidden tests are unavailable and correctness must be estimated through imperfect proxies. Beyond increasing the chance of sampling correct code, the candidate set exposes a comparative structure over the model’s plausible attempts.

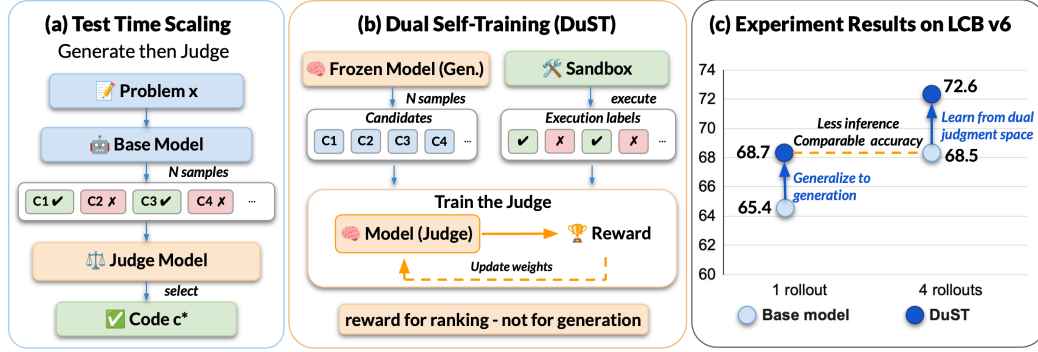


Figure 1: Overview. (a) Conventional test-time scaling samples N candidates and applies a judge, incurring $O(N)$ cost without improving the generator. (b) We propose to self-train from the dual judgment space: the model’s own candidates are executed, and the model is trained via GRPO to rank them by correctness—using only a discriminative ranking reward. (c) Using Qwen3-30B-Thinking on LiveCodeBench v6, our proposed method improves the accuracy of test-time scaling via enhancing the judgment quality and surprisingly generalizing to generation.

Existing generate-then-judge systems typically use this comparative structure only for inference-time ranking, whether through execution feedback, an external judge, or a separately trained reward model [Li et al., 2022, Zhang et al., 2024, Toshniwal et al., 2026, Ma et al., 2026]. Once a candidate is selected, however, the signal is discarded. A strong judge can improve the final answer chosen from the current candidate pool, but it does not directly improve the generator that produced those candidates. The model may therefore continue to assign high probability to the same plausible but incorrect code in future rollouts.

This motivates our primal–dual view of test-time scaling. We use the *primal generation space* to refer to the model’s ordinary code-generation behavior: sampling candidate codes and receiving execution outcomes on them. We use the *dual judgment space* to refer to the induced ranking problem over those candidates: deciding which plausible attempts are correct, which fail, and what distinguishes them. In the primal space, standard RL training receives only a sparse execution reward for each sampled code attempt, often a single binary pass/fail bit. In the dual space, a set of N candidates can induce up to $O(N^2)$ pairwise comparisons, and every mixed group provides multiple correct-versus-incorrect contrasts. The resulting supervision is therefore much denser than the primal one-bit reward by exposing a richer correctness structure across plausible code attempts rather than an isolated success or failure for each attempt.

We introduce **DuST** (Dual Self-Training), a framework for self-training from this dual judgment space. DuST samples candidate codes from the model’s own distribution, labels them through sandbox execution, retains groups containing both successes and failures, and trains the same model to rank candidates by execution correctness using GRPO [Shao et al., 2024]. The objective is purely discriminative: the model is never directly rewarded for generating correct code. Instead, all supervision comes from judging correctness contrasts among its own successful and failed attempts. Our central hypothesis is that a model trained to discriminate correct from incorrect code in the dual judgment space can acquire correctness-sensitive features that benefit generation in the primal space.

Our experiments support this hypothesis. On LiveCodeBench v6 [Jain et al., 2024], Qwen3-30B-Thinking improves by 6.2 NDCG points in judgment quality and by 4.1 points in Best-of-4 test-time scaling accuracy. More importantly, the same training improves single-sample generation by 3.1 pass@1 points, despite using no direct generation reward. This gain generalizes across four base models spanning two model families and three scales (4B to 30B). To isolate the mechanism behind this transfer, we compare GRPO with supervised fine-tuning on the same ranking data. SFT improves judgment but does not improve generation, while GRPO improves both, showing that on-policy reinforcement learning transfers dual-space judgment learning back into primal generation.

2 Method

We now describe DuST, a training procedure that converts multi-candidate code generation and judgment into dual-space supervision. Starting from a base code model, DuST samples candidate

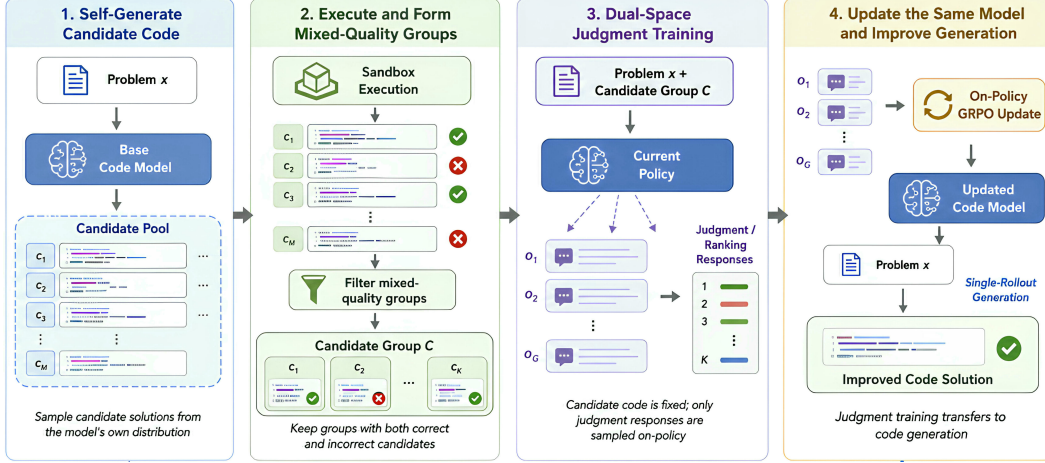


Figure 2: Overview of our framework for self-training from dual judgment space. Starting from a base code model, DuST samples candidate code, executes each candidate to obtain binary correctness labels, forms mixed-quality candidate groups, and trains the same model to rank candidates by execution correctness via GRPO. The sampled code is held fixed during training, and on-policy sampling applies only to the model’s judgment responses. Thus, the reward is applied in the dual judgment space, while the updated parameters are the same ones later used for primal code generation.

code, executes each candidate to obtain binary correctness labels, forms mixed-quality candidate groups, and trains the same model to rank candidates by execution correctness. The sampled code is held fixed during training, and on-policy sampling applies only to the model’s judgment responses. Thus, the reward is applied in the dual judgment space, while the updated parameters are the same ones later used for primal code generation. Figure 2 summarizes the framework.

2.1 Test-Time Scaling: From Primal Generation to Dual Judgment

For a coding problem x , a common test-time scaling pipeline proceeds in two stages. First, the model samples a set of candidate code samples

$$C_N = (c_1, \dots, c_N), \quad c_i \sim \pi_{\text{base}}(\cdot | x).$$

Second, it applies a judge J to choose a final answer,

$$c^* = J(x, C_N).$$

We refer to this two-stage pipeline as *generate-then-judge*. The two stages play distinct roles. Sampling C_N expands coverage in the primal generation space, while the judge defines a dual judgment problem over the sampled candidates by deciding which plausible code attempt should be trusted. In standard test-time scaling, however, this comparative correctness information is consumed only at inference time: the judge selects from the current candidate pool, but the signal does not feed back to improve the generator itself.

We instead reuse generate-then-judge as a source of training supervision. Rather than discarding execution-grounded contrasts after selection, **DuST** converts them into dual-space training signals that improve both the model’s judgment and its generation, so that a single rollout from the trained model approaches the accuracy of the base model’s generate-then-judge pipeline.

2.2 Constructing Dual-Space Data from Self-Generated Code

We construct dual-space training data from the model’s own sampled code. For each problem x , we sample a large candidate pool from the base model,

$$\tilde{C}_M = (\tilde{c}_1, \dots, \tilde{c}_M), \quad \tilde{c}_j \sim \pi_{\text{base}}(\cdot | x).$$

Here M denotes the size of the sampled pool used for data construction, which is distinct from the test-time scaling budget N . These candidates are not external demonstrations. They are code samples

the model itself considers plausible. The resulting supervision is therefore grounded in what the model can actually generate, rather than on arbitrary positives and negatives.

After execution, each candidate receives a binary correctness label

$$\tilde{y}_j = V(x, \tilde{c}_j) \in \{0, 1\},$$

where $V(x, c) = 1$ indicates that candidate code c passes all tests for problem x , and $V(x, c) = 0$ otherwise. We partition the sampled pool into groups of K candidates,

$$C = (c_1, \dots, c_K),$$

with corresponding labels

$$y = (y_1, \dots, y_K), \quad y_i = V(x, c_i),$$

and keep only mixed-quality groups satisfying

$$0 < \sum_{i=1}^K y_i < K.$$

Each retained group therefore contains at least one correct and one incorrect candidate.

This filtering is useful for two reasons. First, it removes groups that contain no ranking contrast: all-correct groups contain no mistakes to distinguish, while all-incorrect groups contain no execution-confirmed correct code to rank above the failures. Second, it concentrates training on cases where the model can sometimes solve the problem, but cannot reliably do so in every sample. Mixed groups are therefore the basic units of dual-space supervision. They expose objective contrasts between what the model can generate correctly and what it finds plausible but incorrect.

This procedure defines a training set

$$\mathcal{D} = \{(x, C, y)\},$$

where each instance contains a problem, a self-generated candidate set, and execution-grounded correctness labels. This dataset is constructed once from the base model and reused throughout training. On-policy sampling during GRPO applies only to the model’s judgment responses, not to the candidate code samples being judged, making the data pipeline a one-time cost.

2.3 Dense Execution-Grounded Ranking Rewards

Given a mixed-quality group, the dual-space objective is to rank execution-correct candidates above execution-incorrect candidates. For a training instance (x, C, y) , where $C = (c_1, \dots, c_K)$, the model receives the problem and candidate code samples and outputs a judgment response o containing an optional reasoning trace followed by a final ranking of candidates. Let

$$\hat{\rho}(o) = (\rho_1, \dots, \rho_K)$$

denote the parsed ranking from best to worst, where earlier positions indicate higher predicted correctness.

We use binary execution labels rather than fractional test-pass rates. In code generation, candidate code that fails an edge case is still incorrect, even if it passes most visible tests. Fractional scores can also reflect properties of the test suite rather than the code itself, such as how many tests cover a particular failure mode. Binary correctness gives a sharper supervision signal: the model must distinguish fully correct code from code that contains any latent bug.

We score only the final parsed ranking, not the intermediate reasoning. This keeps the supervision outcome-based: the model may use whatever reasoning strategy it finds useful, but reward is determined solely by whether its final judgment agrees with execution.

We define the set of execution-distinguishable correct-over-incorrect pairs as

$$\mathcal{P}(y) = \{(i, j) : y_i = 1, y_j = 0\}.$$

Since every retained group contains at least one correct and one incorrect candidate, we have $|\mathcal{P}(y)| > 0$. Let $\text{pos}_{\hat{\rho}}(i)$ denote the position of candidate i in the predicted ranking. We define the execution-grounded ranking reward as

$$R_{\text{exec}}(\hat{\rho}; y) = \frac{1}{|\mathcal{P}(y)|} \sum_{(i,j) \in \mathcal{P}(y)} \mathbb{I}[\text{pos}_{\hat{\rho}}(i) < \text{pos}_{\hat{\rho}}(j)].$$

This reward measures how often correct candidates are ranked above incorrect candidates. Candidates with identical execution labels are not compared, since execution does not specify a preference between them.

To handle malformed judgment responses, we define a response-level reward

$$R(o; y) = \begin{cases} R_{\text{exec}}(\hat{\rho}(o); y), & \text{if } o \text{ contains a valid ranking,} \\ R_{\text{fmt}}, & \text{otherwise,} \end{cases}$$

where $R_{\text{fmt}} < 0$ is a format penalty.

This listwise formulation provides denser feedback than primal pass/fail supervision. Instead of rewarding each candidate independently, the ranking reward exposes all execution-distinguishable correct-over-incorrect contrasts in the group. With $n_+ = \sum_i y_i$ correct and $n_- = K - n_+$ incorrect candidates per group, the dual reward exposes $n_+ n_-$ comparisons, naturally ignoring ties induced by binary labels.

2.4 On-Policy Dual-Space Self-Training

The final step is to make the dual judgment signal update the same policy that will later generate code. The candidate code remains fixed, but the judgment responses are sampled from the current policy.

We use Group Relative Policy Optimization (GRPO) [Shao et al., 2024]. For each training instance $(x, C, y) \sim \mathcal{D}$, we sample a group of G judgment responses from the current policy,

$$o_1, \dots, o_G \sim \pi_{\theta_{\text{old}}}(\cdot \mid x, C),$$

score each with the execution-grounded reward $r_i = R(o_i; y)$, and normalize within the group to obtain relative advantages $A_i = (r_i - \bar{r}) / (\text{std}(r_1, \dots, r_G) + \epsilon)$. The policy is then updated by maximizing the standard clipped GRPO objective with KL regularization against the frozen reference policy π_{ref} , using clip ratio ϵ and KL coefficient β .

This on-policy step distinguishes DuST from supervised imitation of rankings. A supervised model can learn to reproduce correct rankings, but imitation alone need not change the distribution from which code is generated. GRPO instead samples the model’s own judgment responses, scores them with execution-grounded rewards, and updates the same parameters used for generation. As a result, correctness-sensitive features learned in the dual judgment task can alter the model’s primal generation behavior.

This creates the intended path from dual judgment back to primal generation. To rank candidates correctly, the model must learn features associated with executable correctness, including algorithmic invariants, edge cases, and failure modes. Because these features are learned by the generator itself, they can shift probability mass away from flawed code completions and toward correct code. Thus, any generation improvement is not the result of direct generation supervision, but of the dual judgment signal transferred into the model’s generative behavior through on-policy RL.

3 Experiments

3.1 Experimental Setup

We apply DuST to four models spanning two families (Qwen, GPT-oss), three scales (4B–30B), and two reasoning styles (instruct, thinking): Qwen3-4B-Thinking, Qwen3-30B-Instruct, Qwen3-30B-Thinking, and GPT-oss-20B. Training data is sourced from the seed subset of rSTARcoder [Liu et al., 2025], de-duplicated to yield $\sim 10\text{K}$ unique competitive programming problems. For each problem, we sample $N=64$ candidates from the base model and execute them in a sandboxed environment to obtain binary correctness labels. We partition candidates into groups of $K=4$ and retain only mixed-quality groups (at least one correct and one incorrect), yielding $\sim 6.9\text{K}$ valid queries and 37K training groups for the primary model; among retained groups, 17.8% contain one correct candidate, 24.9% two, and 57.3% three.

We fine-tune with GRPO [Shao et al., 2024] using the veRL framework for 3 epochs. Learning rates are 1×10^{-6} for Qwen models and 5×10^{-7} for GPT-oss-20B; rollout group size is $G=8$ with 3 PPO epochs per batch. MoE models train on 32 B200 GPUs with Megatron-based parallelism; dense

Table 1: Test-time scaling performance (accuracy %, mean over 10 repeats) after generating 4 candidates and selecting the top-ranked one, on LiveCodeBench v6 (Feb–May 2025) and v5 (Aug 2024–Feb 2025), stratified by difficulty. Top section lists existing open-source reasoning models sorted by LCB v6 overall score. Bottom section shows our method applied to each base model, with gains over the base shown in parentheses.

Model	LiveCodeBench v6				LiveCodeBench v5			
	Easy	Med.	Hard	All	Easy	Med.	Hard	All
<i>Existing open-source reasoning models</i>								
OpenReasoning-Nemotron-32B	98.4	73.1	48.0	67.4	97.6	82.8	53.0	73.2
TinyR1-32B	89.5	74.4	46.3	64.9	88.1	85.6	51.1	71.2
OpenCodeReasoning-1.1-32B	97.6	71.2	44.3	64.9	95.4	83.4	49.4	71.3
OpenReasoning-Nemotron-14B	97.6	72.4	39.3	63.0	95.7	83.4	46.8	70.2
OpenCodeReasoning-1.1-14B	96.8	65.4	38.5	60.3	95.4	76.6	43.3	66.6
OpenReasoning-Nemotron-7B	99.2	66.0	34.0	59.0	97.0	78.0	43.2	67.3
Klear-Reasoner-8B	98.4	64.7	32.4	57.6	96.4	77.3	35.1	63.3
DeepSeek-R1-0528-Qwen3-8B	97.6	56.4	29.9	53.8	96.5	72.6	30.1	59.6
OpenCodeReasoning-1.1-7B	94.4	55.8	13.1	45.0	94.6	60.8	17.2	49.7
Skywork-OR1-7B	89.5	41.0	15.2	40.5	88.1	52.2	20.0	46.8
<i>Our method vs. base model</i>								
Qwen3-30B-Thinking	100.0	73.7	47.5	68.7	98.6	84.4	49.2	72.3
+ DuST	100.0	82.3	52.5	72.6 (+3.9)	98.6	87.0	54.1	75.2 (+2.9)
Qwen3-30B-Instruct	82.9	49.2	19.0	43.1	92.4	47.4	19.3	46.0
+ DuST	88.2	49.6	24.6	47.1 (+4.0)	90.2	53.5	18.7	47.1 (+1.1)
GPT-OSS-20B	98.2	75.2	43.5	65.2	97.0	79.3	42.9	67.5
+ DuST	98.7	78.5	47.1	69.4 (+4.2)	97.5	81.4	53.1	72.8 (+5.3)
Qwen3-4B-Thinking	97.6	60.3	29.9	55.0	95.9	73.3	36.1	62.3
+ DuST	98.4	68.0	34.0	59.4 (+4.4)	97.9	77.8	36.9	64.6 (+2.3)

models on 8 to 16 B200 GPUs with FSDP. We evaluate on LiveCodeBench v6 (LCB v6, Feb–May 2025) [Jain et al., 2024] as the primary benchmark and LCB v5 (Aug 2024–Feb 2025) as secondary confirmation, reporting Best-of-4 accuracy and NDCG averaged over 10 repeats. We compare against 10 strong open-source reasoning models (Table 1). More implementation details and prompts are in Appendix B.

3.2 Main Results

Table 1 compares our method against existing open-source reasoning models on LiveCodeBench v6 and v5. Across all four base models, our method consistently improves test-time scaling accuracy. The gains are substantial: Qwen3-30B-Thinking improves from 68.7% to 72.6% (+3.9) on LCB v6 and from 72.3% to 75.2% (+2.9) on LCB v5. Qwen3-4B-Thinking improves from 55.0% to 59.4% (+4.4) on LCB v6, Qwen3-30B-Instruct improves from 43.1% to 47.1% (+4.0), and GPT-OSS-20B improves from 65.2% to 69.4% (+4.2) on LCB v6. The improvements concentrate on medium and hard problems, where execution-grounded judgment is most valuable: for Qwen3-30B-Thinking on LCB v6, medium accuracy improves from 73.7% to 82.3% and hard from 47.5% to 52.5%, while easy problems are already saturated near 100%. The gains generalize across model families (Qwen and GPT), reasoning styles (thinking and instruct), and scales (4B to 30B), suggesting that execution-grounded ranking reward provides a general-purpose training signal rather than one specific to a particular architecture or scale.

Dual Training Improves Both Judgment and Generation To understand where the test-time scaling improvements originate, we decompose the effect of training into generation quality (pass@1), judgment quality (NDCG), and test-time scaling (accuracy) under three configurations: *Baseline* (untrained base model for both generator and ranker), *+ Dual-Trained Judge* (trained ranker, untrained base generator), and *+ Dual-Trained Gen. & Judge* (single model for both, trained only with dual judgement training).

Table 2: Decomposing the effect of dual training on generation (pass@1), judgment (NDCG), and test-time scaling (accuracy). Each row adds one trained component. The gap between rows 2 and 3 isolates the duality effect: the same dual training that improves judgment also improves generation, though generation was never rewarded.

Configuration	LiveCodeBench v6			LiveCodeBench v5		
	Generation pass@1	Judgment NDCG	TTS Accuracy	Generation pass@1	Judgment NDCG	TTS Accuracy
<i>Qwen3-30B-Thinking</i>						
Baseline	65.4	70.1±0.3	68.7	69.2	76.0±0.1	72.3
+ Dual-Trained Judge	65.4	73.7±0.3 (+3.6)	70.4 (+1.7)	69.9	76.7±0.1 (+0.7)	74.3 (+2.0)
+ Dual-Trained Gen. & Judge	68.5 (+3.1)	76.3 ±0.3 (+6.2)	72.6 (+3.9)	71.0 (+1.8)	78.2 ±0.2 (+2.2)	75.2 (+2.9)
<i>Qwen3-30B-Instruct</i>						
Baseline	41.1	47.2±0.5	43.1	44.0	50.7	46.0
+ Dual-Trained Judge	41.1	48.9±1.0 (+1.7)	45.2 (+2.1)	44.0	51.3 (+0.6)	46.8 (+0.8)
+ Dual-Trained Gen. & Judge	42.5 (+1.4)	49.4 ±0.6 (+2.2)	47.1 (+4.0)	44.5 (+0.5)	51.6 (+0.9)	47.1 (+1.1)
<i>GPT-OSS-20B</i>						
Baseline	62.7	70.7±0.2	65.2	67.8	74.9±0.1	67.5
+ Dual-Trained Judge	63.5	72.0±0.6 (+1.3)	68.6 (+3.4)	67.7	76.1 (+1.2)	71.7 (+4.2)
+ Dual-Trained Gen. & Judge	65.0 (+2.3)	72.0 ±0.3 (+1.3)	69.4 (+4.2)	69.8 (+2.0)	76.9 (+2.0)	72.8 (+5.3)
<i>Qwen3-4B-Thinking</i>						
Baseline	45.8	60.5±0.6	55.0	52.9	67.2±0.3	62.3
+ Dual-Trained Judge	47.0 (+1.2)	63.1±0.9 (+2.6)	58.8 (+3.8)	52.4	68.0±0.2 (+0.8)	63.9 (+1.6)
+ Dual-Trained Gen. & Judge	49.1 (+3.3)	63.0 ±0.4 (+2.5)	59.4 (+4.4)	56.3 (+3.4)	68.5 ±0.2 (+1.3)	64.6 (+2.3)

Table 2 reports the decomposition across all four trained models. Two findings stand out. First, dual judgment training consistently improves NDCG across all models (e.g., +6.2 for Qwen3-30B-Thinking, +2.6 for Qwen3-4B-Thinking on LCB v6). Second, the gap between rows 2 and 3 isolates the duality effect: the same dual training that improves judgment also improves primal generation quality despite no direct reward on code correctness. Pass@1 improves for every model (e.g., 65.4%→68.5% for Qwen3-30B-Thinking, 62.7%→65.0% for GPT-OSS-20B), and the gains from better judgment and better generation compound in the *Dual-Trained Gen. & Judge* configuration. We hypothesize that this transfer occurs because ranking requires the model to internalize features of executable correctness, like algorithmic invariants, edge cases, implementation bugs. These same features shift the generator toward correct trajectories.

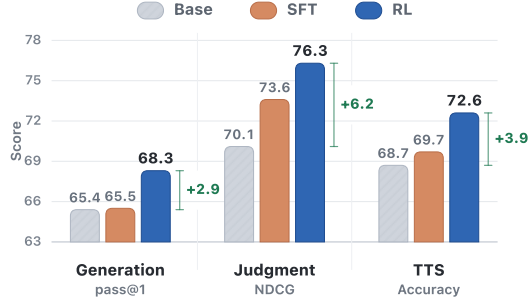


Figure 3: Comparison of training modes on LCB v6 (Qwen3-30B-Thinking). SFT improves judgment but not generation; RL improves all three. Here, TTS is short for test time scaling.

Dual Ranking Training Outperforms Primal Generation Training We compare four RL training modes, all using the same base model and coding prompts (Table 3). **Off-policy ranking only** (our default) trains in the dual space by ranking pre-generated candidates fixed throughout training. **On-policy ranking only** replaces these with candidates sampled from the current policy. **On-policy Generation only** removes ranking and trains purely in the primal space with binary execution reward (pass/fail). **Iterative (gen+rank)** applies generation and ranking updates at 50 steps iteratively.

The key finding is that dual ranking training outperforms primal generation training across all dimensions. Off-policy ranking achieves the best performance (72.6% accuracy, 68.3% pass@1), while generation-only reaches only 71.4% accuracy and 67.1% pass@1. This is notable because ranking never directly rewards the model for producing correct code, yet it yields stronger generation than direct execution feedback. The primal binary pass/fail reward is sparser than the dual ranking reward, which exposes the contrastive structure between correct and incorrect solutions through pairwise comparisons per instance.

RL Drives the Dual-to-Primal Transfer To isolate whether the transfer from dual judgment to primal generation requires RL’s on-policy mechanism or arises from task synergy alone, we compare

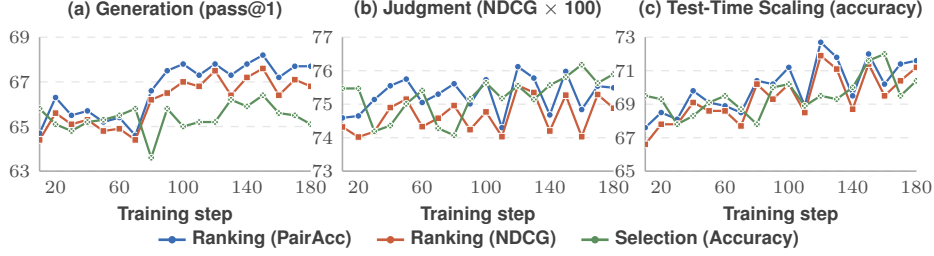


Figure 4: Training dynamics under different judge rewards (Qwen3-30B-Thinking, LiveCodeBench v6). Ranking rewards (PairAcc and NDCG) achieve better generation transfer (pass@1) than Selection reward, which improves judgment but transfers less to generation.

GRPO against an SFT baseline trained on the same ranking data using Qwen3-235B-Thinking teacher rankings (pairwise accuracy = 1.0), with identical prompt format and data distribution. The SFT model is trained with learning rate 1×10^{-5} , 3 epochs, and maximum sequence length 65,536. Figure 3 compares Base, SFT, and RL (GRPO) across all three dimensions. SFT improves judgment (NDCG: 70.1 $\rightarrow$ 73.6) and, through better candidate selection from the unchanged base generator, modestly improves accuracy (68.5 $\rightarrow$ 70.4). However, SFT leaves generation essentially unchanged (pass@1: 65.4). RL improves all three simultaneously, generation (65.4 $\rightarrow$ 68.5), judgment (70.1 $\rightarrow$ 76.3), and accuracy (68.5 $\rightarrow$ 72.6), because the on-policy feedback loop reshapes the generator as well. We additionally test GRPO after warmup with SFT: RL recovers generation quality (pass@1: 66.9%, accuracy: 71.8%), but pure GRPO from base still leads, suggesting SFT initialization constrains RL exploration. These results confirm that task synergy alone is insufficient. GRPO’s closed feedback loop between on-policy sampling, execution feedback, and parameter updates, absent in supervised imitation, is what transfers dual correctness understanding into primal generative behavior.

Off-Policy Ranking Outperforms On-Policy Ranking Among ranking variants, off-policy outperforms on-policy (72.6% vs 72.1%) because the pre-generated pool of $N=64$ candidates per problem provides richer contrastive signal, especially for hard problems. On-policy sampling with fewer rollouts is less likely to produce informative mixed-quality groups. The iterative mode suffers from training instability. Generation updates destabilize training, degrading candidate quality and creating a negative feedback loop that causes early collapse.

Table 3: Four RL training modes on LCB v6 (Qwen3-30B-Thinking), all using different rewards. Off-policy ranking (DuST) achieves the best performance across evaluation dimensions. Here, TTS is short for test time scaling. Gen. means generation and Judge. means judgment.

Training Method	Gen. pass@1	Judge. NDCG	TTS Accuracy
Off-policy Ranking	68.3	76.3	72.6
On-policy Ranking	67.2	75.3	72.1
On-policy Generation	67.1	74.6	71.4
Iterative (gen+rank)	66.9	72.4	71.4

Denser Dual Rewards Transfer Better to Generation We compare three reward functions for the ranking task (Figure 4). **Pairwise Accuracy** rewards correct-over-incorrect orderings averaged over all pairs. **NDCG** applies position-dependent discounting to the same binary labels. **Selection** rewards only if the top-ranked candidate is correct. Under binary labels, Pairwise Accuracy and NDCG show similar trends, with Pairwise Accuracy achieving a slight edge (72.7% vs 71.9% peak accuracy). We adopt it as our default for simplicity. Selection achieves comparable ranking quality but transfers less to generation: pass@1 stays flat around 65% to 66% versus the steady climb to 68% with ranking rewards. We attribute this to reward informativeness: ranking requires ordering *all* candidates, exposing the full dual contrastive structure between correct and incorrect code, whereas selection only requires identifying a single best candidate, providing a coarser signal that captures less of this structure.

4 Related Work

Test-time scaling for code generation. Test-time scaling improves model performance by spending extra inference compute on sampling, search, or iterative refinement [Snell et al., 2025, Yuan et al.,

2024, Zhang et al., 2025, Li et al., 2025a, Zhou et al., 2025, Lee et al., Wu et al., 2025a, Muennighoff et al., 2025, Wu et al., 2025b]. In code generation, this is often instantiated as generate-then-judge: multiple candidate programs are sampled and then ranked or selected using execution feedback, tests, or verifiers, with recent methods further improving diversity, comparison, and repair [Ma et al., 2025, Yu et al., 2025b, Aggarwal et al., 2025, Li et al., 2025b, Wang et al.]. However, these correctness signals are consumed at inference time without enriching the model’s primal training signal. DuST instead converts them into dual-space supervision, teaching the model to discriminate among its own verified candidates and transferring judgment signals back into primal code generation.

Reinforcement learning for code generation. Reinforcement learning has been widely used to improve code generation by optimizing execution-grounded signals that are not directly captured by next-token prediction, such as compilation success, unit-test pass rates, functional correctness, or feedback from learned critics and verifiers [Le et al., 2022, Shojaee et al.]. Recent work extends this idea by using execution feedback for multi-turn refinement, constructing denser process-level rewards, or training critics and reward models to guide program repair and candidate selection [Zeng et al., 2025, Gehring et al., 2025, Ye et al., 2025a, Li et al., 2025c, Xie et al., 2025]. These methods typically optimize models to generate better code, revise code using feedback, or provide verification signals. In contrast, DuST trains exclusively in the dual judgment space: execution labels correctness contrasts among the model’s own candidates, and GRPO optimizes a discriminative ranking objective—never directly rewarding primal generation. We find that this dual-space training transfers back into primal generation, improving single-rollout code quality alongside test-time scaling accuracy.

Synergy of generation and judgment. Generation and judgment have become deeply intertwined in preference-based language model training. RLHF and DPO-style methods use human or AI preferences to optimize generators [Ouyang et al., 2022, Rafailov et al., 2023, Yuan et al., 2023, Meng et al., 2024], while self-improvement methods use model-generated critiques, revisions, or rewards as feedback [Bai et al., 2022, Yuan et al., 2024, Wu et al.]. These works show that judgment can guide generation, but their feedback is typically external, subjective, or model-generated. Conversely, recent judge-training methods use generation to improve judgment, asking models to produce rationales, critiques, plans, or reasoning traces before assigning scores [Saha et al., Song et al., 2024, Ye et al., 2025b, Yu et al., 2025a]. In contrast, DuST asks whether judgment learning itself can become a route to better generation. We study this question in code generation, where judgment can be grounded in objective execution outcomes. This distinguishes DuST from preference optimization, which aligns generation to external preferences; from judge-training work, which primarily improves evaluation; and from standard code RL, which directly optimizes program success. Our focus is whether discriminative judgment learning can transfer back to primal code generation.

5 Conclusion

We introduced DuST, a framework that self-trains code models in the dual judgment space exposed by test-time scaling. Rather than learning from isolated pass/fail feedback in the primal space, DuST trains the model on-policy to discriminate correct from incorrect candidates—never directly rewarding generation. The core premise is that generation and judgment are dual: a model that learns to discriminate correct from incorrect code should also produce better code, since understanding what makes a solution succeed informs how to generate one. Our experiments confirm this across four models: DuST consistently improves judgment, generation, and test-time scaling accuracy, with a single rollout from the trained model matching the base model’s Best-of-4 performance. SFT on the same data improves judgment but not generation, confirming that on-policy RL is the mechanism that transfers dual-space learning back into primal generation.

These findings suggest that the primal and dual spaces of code generation are more tightly coupled than standard training assumes. The contrastive correctness structure exposed at test time—which trajectories succeed, which fail, and what distinguishes them—is a rich and largely untapped training signal. Exploiting this duality more broadly, through iterative self-improvement, harder problem curricula, or richer dual objectives, is a promising direction for future work.

References

Vaibhav Aggarwal, Ojasv Kamal, Abhinav Japesh, Zhijing Jin, and Bernhard Schölkopf. Dars: Dynamic action re-sampling to enhance coding agent performance by adaptive tree traversal. In *Proceedings of the*

330 *63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages
331 19808–19855, 2025.

332 Wasi Uddin Ahmad, Somshubra Majumdar, Aleksander Ficek, Sean Narenthiran, Mehrzad Samadi, Jocelyn
333 Huang, Siddhartha Jain, Vahid Noroozi, and Boris Ginsburg. OpenCodeReasoning-II: A Simple Test Time
334 Scaling Approach via Self-Critique, 2025a. URL <https://arxiv.org/abs/2507.09075>.

335 Wasi Uddin Ahmad, Sean Narenthiran, Somshubra Majumdar, Aleksander Ficek, Siddhartha Jain, Jocelyn
336 Huang, Vahid Noroozi, and Boris Ginsburg. Opencodereasoning: Advancing data distillation for competitive
337 coding. 2025b. URL <https://arxiv.org/abs/2504.01943>.

338 Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen,
339 Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback.
340 *arXiv preprint arXiv:2212.08073*, 2022.

341 DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *CoRR*,
342 abs/2501.12948, 2025. doi: 10.48550/ARXIV.2501.12948. URL [https://doi.org/10.48550/arXiv.](https://doi.org/10.48550/arXiv.2501.12948)
343 [2501.12948](https://doi.org/10.48550/arXiv.2501.12948).

344 Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Taco Cohen, and Gabriel Synnaeve. Rlef: Grounding
345 code llms in execution feedback with reinforcement learning. In *International Conference on Machine*
346 *Learning*, pages 19034–19055. PMLR, 2025.

347 Jujie He, Jiakai Liu, Chris Yuhao Liu, Rui Yan, Chaojie Wang, Peng Cheng, Xiaoyu Zhang, Fuxiang Zhang,
348 Jiacheng Xu, Wei Shen, Siyuan Li, Liang Zeng, Tianwen Wei, Cheng Cheng, Bo An, Yang Liu, and Yahui
349 Zhou. Skywork open reasoner 1 technical report. *CoRR*, abs/2505.22312, 2025. doi: 10.48550/ARXIV.2505.
350 [22312](https://doi.org/10.48550/arXiv.2505.22312). URL <https://doi.org/10.48550/arXiv.2505.22312>.

351 Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama,
352 Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language
353 models for code. *arXiv preprint arXiv:2403.07974*, 2024.

354 Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. Coderl: Mastering
355 code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information*
356 *Processing Systems*, 35:21314–21328, 2022.

357 Hyunseok Lee, Seunghyuk Oh, Jaehyung Kim, Jinwoo Shin, and Jihoon Tack. Revise: Learning to refine at
358 test-time via intrinsic self-verification. In *Forty-second International Conference on Machine Learning*.

359 Cheryl Li, Tianyuan Xu, and Steven Y Guo. Reasoning-as-logic-units: Scaling test-time reasoning in large
360 language models through logic unit alignment. In *International Conference on Machine Learning*, pages
361 36530–36550. PMLR, 2025a.

362 Dacheng Li, Shiyi Cao, Chengkun Cao, Xiuyu Li, Shangyin Tan, Kurt Keutzer, Jiarong Xing, Joseph E Gonzalez,
363 and Ion Stoica. S*: Test time scaling for code generation. *arXiv preprint arXiv:2502.14382*, 2025b.

364 Qingyao Li, Xinyi Dai, Xiangyang Li, Weinan Zhang, Yasheng Wang, Ruiming Tang, and Yong Yu. Codeprm:
365 Execution feedback-enhanced process reward model for code generation. In *Findings of the Association for*
366 *Computational Linguistics: ACL 2025*, pages 8169–8182, 2025c.

367 Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James
368 Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor
369 Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy,
370 Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and
371 Oriol Vinyals. Competition-level code generation with alphacode. *arXiv preprint arXiv:2203.07814*, 2022.

372 Yifei Liu, Li Lyna Zhang, Yi Zhu, Bingcheng Dong, Xudong Zhou, Ning Shang, Fan Yang, and Yang Mao. rstar-
373 coder: Scaling competitive code reasoning with a large-scale verified dataset. *arXiv preprint arXiv:2505.21297*,
374 2025.

375 Yingwei Ma, Yongbin Li, Yihong Dong, Xue Jiang, Yanhao Li, Yue Liu, Rongyu Cao, Jue Chen, Fei Huang,
376 and Binhua Li. Thinking longer, not larger: Enhancing software engineering agents via scaling test-time
377 compute. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*,
378 pages 3730–3741. IEEE, 2025.

379 Zeyao Ma, Jing Zhang, Xiaokang Zhang, Jiayi Yang, Zongmeng Zhang, Jiajun Zhang, Yuheng Jing, Lei Zhang,
380 Hao Zheng, Wenting Zhao, Junyang Lin, and Binyuan Hui. Scaling agentic verifier for competitive coding.
381 *arXiv preprint arXiv:2602.04254*, 2026.

382 Yu Meng, Mengzhou Xia, and Danqi Chen. Simpo: Simple preference optimization with a reference-free reward.
383 *Advances in Neural Information Processing Systems*, 37:124198–124235, 2024.

384 Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer,
385 Percy Liang, Emmanuel Candès, and Tatsunori B Hashimoto. s1: Simple test-time scaling. In *Proceedings of*
386 *the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20286–20332, 2025.

387 Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang,
388 Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with
389 human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

390 Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct
391 preference optimization: Your language model is secretly a reward model. *Advances in neural information*
392 *processing systems*, 36:53728–53741, 2023.

393 Swarnadeep Saha, Xian Li, Marjan Ghazvininejad, Jason Weston, and Tianlu Wang. Learning to plan & reason
394 for evaluation with thinking-llm-as-a-judge, 2025. URL <https://arxiv.org/abs/2501.18099>.

395 Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang,
396 Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open
397 language models. *arXiv preprint arXiv:2402.03300*, 2024.

398 Parshin Shojaei, Aneesh Jain, Sindhu Tipirneni, and Chandan K Reddy. Execution-based code generation using
399 deep reinforcement learning. *Transactions on Machine Learning Research*.

400 Jianfeng Si, Lin Sun, Zhewen Tan, and Xiangzheng Zhang. Efficient switchable safety control in llms via
401 magic-token-guided co-training, 2025. URL <https://arxiv.org/abs/2508.14904>.

402 Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally
403 can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on*
404 *Learning Representations*, 2025.

405 Yuda Song, Hanlin Zhang, Carson Eisenach, Sham M Kakade, Dean Foster, and Udaya Ghai. Mind the gap:
406 Examining the self-improvement capabilities of large language models. In *The Thirteenth International*
407 *Conference on Learning Representations*, 2024.

408 Zhenpeng Su, Lei Yu Pan, Xue Bai, Dening Liu, Guanting Dong, Jiaming Huang, Wenping Hu, Fuzheng
409 Zhang, Kun Gai, and Guorui Zhou. Klear-reasoner: Advancing reasoning capability via gradient-preserving
410 clipping policy optimization. *CoRR*, abs/2508.07629, 2025. doi: 10.48550/ARXIV.2508.07629. URL
411 <https://doi.org/10.48550/arXiv.2508.07629>.

412 TinyR1 Team. Tinyr1-32b-report, 2025.

413 Shubham Toshniwal, Aleksander Ficek, Siddhartha Jain, Wei Du, Vahid Noroozi, Sadegh Mahdavi, Somshubra
414 Majumdar, and Igor Gitman. Learning generative selection for best-of-n. *arXiv preprint arXiv:2602.02143*,
415 2026.

416 Evan Z Wang, Federico Cassano, Catherine Wu, Yunfeng Bai, William Song, Vaskar Nath, Ziwen Han, Sean M
417 Hendryx, Summer Yue, and Hugh Zhang. Planning in natural language improves llm search for code
418 generation. In *The First Workshop on System-2 Reasoning at Scale, NeurIPS’24*.

419 Menghua Wu, Cai Zhou, Stephen Bates, and Tommi Jaakkola. Thought calibration: Efficient and confident test-
420 time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*,
421 pages 14302–14316, 2025a.

422 Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical
423 analysis of compute-optimal inference for llm problem-solving. In *The Thirteenth International Conference*
424 *on Learning Representations*, 2025b.

425 Yue Wu, Zhiqing Sun, Huizhuo Yuan, Kaixuan Ji, Yiming Yang, and Quanquan Gu. Self-play preference
426 optimization for language model alignment. In *The Thirteenth International Conference on Learning*
427 *Representations*.

428 Zhihui Xie, Jie Chen, Liyu Chen, Weichao Mao, Jingjing Xu, and Lingpeng Kong. Teaching language models to
429 critique via reinforcement learning. In *International Conference on Machine Learning*, pages 68559–68577.
430 PMLR, 2025.

431 Yufan Ye, Ting Zhang, Wenbin Jiang, and Hua Huang. Process-supervised reinforcement learning for code
432 generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*,
433 pages 14224–14237, 2025a.

434 Ziyi Ye, Xiangsheng Li, Qiuchi Li, Qingyao Ai, Yujia Zhou, Wei Shen, Dong Yan, and Yiqun Liu. Learning llm-
435 as-a-judge for preference alignment. In *The Thirteenth International Conference on Learning Representations*,
436 2025b.

437 Yue Yu, Zhengxing Chen, Aston Zhang, Liang Tan, Chenguang Zhu, Richard Yuanzhe Pang, Yundi Qian,
438 Xuewei Wang, Suchin Gururangan, Chao Zhang, et al. Self-generated critiques boost reward modeling for
439 language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the*
440 *Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages
441 11499–11514, 2025a.

442 Zhaojian Yu, Yinghao Wu, Yilun Zhao, Arman Cohan, and Xiao-Ping Zhang. Z1: Efficient test-time scaling
443 with code. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing:*
444 *Industry Track*, pages 2688–2712, 2025b.

445 Hongyi Yuan, Zheng Yuan, Chuanqi Tan, Wei Wang, Songfang Huang, and Fei Huang. Rrhf: Rank responses
446 to align language models with human feedback. *Advances in Neural Information Processing Systems*, 36:
447 10935–10950, 2023.

448 Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason
449 Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.

450 Huaye Zeng, Dongfu Jiang, Haozhe Wang, Ping Nie, Xiaotong Chen, and Wenhui Chen. Acecoder: Acing
451 coder rl via automated test-case synthesis. In *Proceedings of the 63rd Annual Meeting of the Association for*
452 *Computational Linguistics (Volume 1: Long Papers)*, pages 12023–12040, 2025.

453 Fuxiang Zhang, Jiacheng Xu, Chaojie Wang, Ce Cui, Yang Liu, and Bo An. Incentivizing llms to self-verify
454 their answers. *arXiv preprint arXiv:2506.01369*, 2025.

455 Lunjun Zhang, Arian Hosseini, Hritik Bansal, Mehran Kazemi, Aviral Kumar, and Rishabh Agarwal. Generative
456 verifiers: Reward modeling as next-token prediction. *arXiv preprint arXiv:2408.15240*, 2024.

457 Yilun Zhou, Austin Xu, Peifeng Wang, Caiming Xiong, and Shafiq Joty. Evaluating judges as evaluators: The
458 jetts benchmark of llm-as-judges as test-time scaling evaluators. In *International Conference on Machine*
459 *Learning*, pages 79436–79471. PMLR, 2025.

A Limitations

We discuss the limitations of this work from four perspectives: the execution-grounded scope of our supervision, the dependence on informative candidate sets, the coverage of our empirical evaluation, and the incomplete understanding of judgment-to-generation transfer.

A.1 Execution-Grounded Scope

DuST relies on execution-grounded correctness labels, which makes the framework especially suitable for code generation but less directly applicable to open-ended generation tasks. A key advantage of code generation is that candidate solutions can be executed in a sandbox and labeled with objective pass/fail outcomes. This enables us to construct dense pairwise correctness contrasts without relying on human preferences or model-generated judgments. However, many generation domains do not admit such clean verification. For tasks such as open-ended writing, dialogue, or complex reasoning without executable checks, correctness is often subjective, underspecified, or only partially observable. Extending the dual self-training idea beyond code therefore requires reliable task-specific verifiers or carefully designed proxy signals. Our results should thus be interpreted as evidence for execution-grounded domains rather than a general claim that discriminative self-training always transfers to generation.

A.2 Dependence on Mixed-Quality Candidate Sets

The effectiveness of DuST depends on the model producing candidate groups that contain both successes and failures. Our training signal comes from correctness contrasts among the model’s own sampled candidates. This makes the method naturally self-adaptive, but it also introduces a dependence on the quality and diversity of the candidate pool. If the base model is too weak, most candidate groups may contain only failures, providing little positive signal for ranking. If the model is already very strong on a task, candidate groups may contain mostly successes, leaving few informative negative examples. The dual judgment space becomes less informative. The method is therefore most effective in a regime where the model can occasionally solve a problem but still produces plausible incorrect attempts, allowing execution feedback to reveal meaningful correctness boundaries.

A.3 Imperfect Verification

Execution feedback is objective but not always a perfect measure of semantic correctness. Sandbox execution provides a stronger supervision signal than free-form preference judgments, but it is still bounded by the quality and coverage of the available tests. A candidate that passes the provided tests may still fail hidden edge cases, rely on brittle assumptions, or exploit accidental properties of the test suite. Conversely, a candidate may fail due to minor formatting or environment issues despite containing largely correct algorithmic logic. Since DuST uses execution outcomes to construct ranking labels, such imperfections can introduce noise into the dual judgment signal. Although our evaluation uses held-out benchmark tests to measure final performance, stronger verification protocols would further reduce the risk that the model learns benchmark-specific or test-specific shortcuts.

A.4 Mechanism and Training Stability

Our results show that GRPO transfers discriminative judgment learning to generation, but the precise mechanism remains only partially understood. The contrast between GRPO and SFT suggests that on-policy reinforcement learning is critical: SFT improves judgment but does not yield the same generative gains, while GRPO improves both. However, this does not fully explain how discriminative ranking updates reshape the model’s generation distribution. The transfer may involve changes in internal correctness features, probability mass shifts away from flawed reasoning trajectories, or improved exploration induced by on-policy optimization. Moreover, RL training can be sensitive to candidate sampling strategy, reward design, group construction, and optimization hyperparameters. A deeper mechanistic analysis of when and why dual-space training improves primal generation remains an important direction for future work.

B Experiment Setup

Data synthesis. We use the seed subset of the rSTARcoder dataset [Liu et al., 2025], de-duplicated to yield $\sim 10K$ unique competitive programming problems. For each prompt, we sample $N=64$ candidate solutions from the base model using vLLM with temperature 1.0, top- p 1.0, top- k 0, and 128K maximum sequence length. Each candidate is executed in a sandboxed environment against the problem’s test suite to obtain a binary correctness label (pass/fail). We apply minimal syntactic filtering to remove empty responses and failed code extractions, meaning there is absolutely no correctness signal used during generation. We then partition the 64 candidates per problem into groups of $K=4$ and retain only *mixed-quality* groups containing at least one correct and one incorrect candidate, discarding trivially solved or unsolved problems. This yields approximately 6.9K valid queries and 37K training groups for the primary model (Qwen3-30B-Thinking). Among the retained groups, 17.8% contain one correct candidate, 24.9% contain two, and 57.3% contain three.

Table 4: Full GRPO training configuration when using Qwen3-30B-A3B-Thinking as the base model. The Mixture-of-Experts model requires expert parallelism.

Parameter	Value
Rollouts per prompt (K)	8
Global batch size	64 prompts $\times$ 8 rollouts = 512
PPO mini-batch	16
PPO micro-batch per GPU	2
PPO epochs per batch	3
Learning rate	1×10^{-6}
KL regularization (β)	0.001
Clip ratio	0.2
Total data epochs	3
Advantage normalization	Group std (GRPO)
Max sequence length	32768 (prompt) + 32768 (response)
Checkpoint frequency	Every 10 steps
<i>Infrastructure</i>	
Actor parallelism	TP=4, PP=1, EP=8, CP=1
vLLM rollout	TP=4, GPU util. 0.7
Hardware	4 nodes $\times$ 8 B200 GPUs = 32 GPUs

Training.

We fine-tune with GRPO [Shao et al., 2024] using the VERL framework¹. The rollout group size is $G=8$, with 3 PPO epochs per batch, clip ratio 0.2, and KL coefficient $\beta=0.001$. We train for 3 epochs over the data. For Qwen models, we use a learning rate 1×10^{-6} and a global batch size of 64 to 128 prompts. For GPT-OSS-20B we use 5×10^{-7} and batch size 32. MoE models (Qwen3-30B variants) use Megatron-based parallelism (EP=8) on 32 B200 GPUs across 4 nodes. The dense Qwen3-4B-Thinking trains on 8 B200 GPUs across 1 node with FSDP, and GPT-OSS-20B trains on 16 B200 GPUs with FSDP2. On-policy rollouts are served by vLLM with temperature 1.0, top- p 1.0, and 0.7 GPU memory utilization.

Evaluation. Our primary benchmark is LiveCodeBench v6 (LCB v6, Feb to May 2025) [Jain et al., 2024], comprising 131 problems stratified by difficulty (31 easy, 39 medium, 61 hard). We report LCB v5 (Aug 2024 to Feb 2025, following the version split adopted by rSTARcoder [Liu et al., 2025]) as a secondary confirmation. For each evaluation run, the generator samples four candidate codes per problem using temperature 0.6, top- p 0.95, top- k 20, and maximum generation length 65,536 tokens. The ranker orders the four candidates, and the top-ranked solution is executed against test cases. We report the mean performance over 10 repeats. Our primary metrics are pass@1 for code generation and NDCG for code ranking. For direct generation, pass@1 measures the probability that a single sample is correct. For test-time scaling, the generator produces four candidate codes, the ranker selects the top-ranked one, and accuracy is computed on the selected solution. We choose 10 strong open-source reasoning models as baselines, as listed in Table 1, including OpenReasoning-Nemotron-32B [Ahmad et al., 2025a], TinyR1-32B [Team, 2025, Si et al., 2025],

¹<https://github.com/verl-project/verl>

OpenCodeReasoning-1.1-32B [Ahmad et al., 2025b], OpenReasoning-Nemotron-14B [Ahmad et al., 2025a], OpenCodeReasoning-1.1-14B [Ahmad et al., 2025b], OpenReasoning-Nemotron-7B [Ahmad et al., 2025a], Klear-Reasoner-8B [Su et al., 2025], DeepSeek-R1-0528-Qwen3-8B [DeepSeek-AI, 2025], OpenCodeReasoning-1.1-7B [Ahmad et al., 2025b], Skywork-OR1-7B [He et al., 2025]. We evaluate those baselines using its optimal parameters using the consistent prompts with our proposed method. For each baseline, we run it for 10 times for the average evaluation scores.

C Prompt Design

C.1 Code Ranking Prompt

The ranking prompt presents the model with a coding problem and K candidate solutions, and asks the model to rank them from most to least likely to pass all tests. The model is free to produce an optional reasoning trace before outputting the final ranking; reward is computed solely on the final parsed ranking.

The prompt template is as follows:

```
You will be given a question (problem specification) and {K}
candidate solutions. You will analyze these candidates and RANK
them from most likely to least likely to pass all tests. You will
NOT return anything except for the ranking.

Question: {problem_description}

{candidate_solutions}

Analyze each candidate solution, analyze all the strengths and
weaknesses, and rank them from most effective to least effective.
Output ONLY the ranking as comma-separated numbers from best to
worst (e.g., "2, 4, 1, 3").

Ranking (best to worst):
```

Each candidate is formatted as:

```
-- CANDIDATE {i} --
“python
{code}
“
```

The ranking output is parsed as a comma-separated sequence of candidate indices. If the output does not contain a valid ranking (e.g., due to format errors or repeated indices), the response receives the format penalty $R_{\text{fmt}} = -1$.

C.2 Code Generation Prompt

During data synthesis and evaluation, the model generates code solutions using the following prompt template. For stdin-type problems:

```
You will be given a question (problem specification) and will
generate a correct Python program that matches the specification
and passes all tests. You will NOT return anything except for the
program.

Question: {question}

Read the inputs from stdin, solve the problem, and write the answer
to stdout (do not directly test on the sample inputs). Enclose
your code within delimiters as follows. Ensure that when the Python
program runs, it reads the inputs, runs the algorithm, and writes
output to STDOUT.
“python
# YOUR CODE HERE
“
```

588 For function-type problems with starter code:

589 You will be given a question (problem specification) and will
590 generate a correct Python program that matches the specification
591 and passes all tests. You will NOT return anything except for the
592 program.

593 Question: {question}

594 You will use the following starter code to write the solution to the
595 problem and enclose your code within delimiters.
596 “python
597 {starter_code}
598 “

599 For thinking models (e.g., Qwen3-30B-Thinking), the instruction “You will NOT return anything
600 except for the program” is omitted, allowing the model to reason before the code block.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction clearly state our main claims: (1) dual-space discriminative training improves both judgment and generation, (2) a single rollout from the trained model matches the base model’s Best-of-4 performance, and (3) on-policy RL is the critical mechanism. All claims are supported by experimental results in Section 3.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Limitations are discussed in Appendix A, including the restriction to competitive programming problems, the reliance on sandbox execution for correctness labels, and the compute requirements for generating large candidate pools.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This paper does not include theoretical results or proofs; all contributions are empirical.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Full training hyperparameters, infrastructure details, data synthesis procedure, evaluation protocol, and prompt templates are provided in Section 3 and Appendix B–C. We will also release code and model checkpoints.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We will release training code, model checkpoints, and the generated training data upon publication.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Training hyperparameters, infrastructure, data synthesis procedure, and evaluation protocol are described in Section 3 and Appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: All reported results are averaged over 10 independent evaluation repeats. Standard deviations are reported for NDCG in the decomposition table (Table 2).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Hardware details (GPU type, number of nodes) and parallelism configurations are provided in Appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research involves only publicly available datasets and open-source models, with no human subjects or sensitive data. We have reviewed the NeurIPS Code of Ethics and confirm compliance.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: This work improves code generation models for competitive programming. We do not foresee direct negative societal impacts beyond those already associated with general-purpose code generation.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released models are fine-tuned for competitive programming and do not pose elevated misuse risks compared to the base models they are derived from.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All datasets (rSTARcoder, LiveCodeBench) and base models (Qwen3 series, GPT-oss-20B) are properly cited. We use these assets in accordance with their respective licenses.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: We release four new fine-tuned model checkpoints (Qwen3-4B-Thinking, Qwen3-30B-Instruct, Qwen3-30B-Thinking, GPT-oss-20B trained with DuST), along with training code, data, and documentation.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

915 Answer: [N/A]
 916 Justification: This paper does not involve human subjects.
 917 Guidelines:
 918 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
 919 with human subjects.
 920 • Depending on the country in which research is conducted, IRB approval (or equivalent)
 921 may be required for any human subjects research. If you obtained IRB approval, you
 922 should clearly state this in the paper.
 923 • We recognize that the procedures for this may vary significantly between institutions
 924 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
 925 guidelines for their institution.
 926 • For initial submissions, do not include any information that would break anonymity (if
 927 applicable), such as the institution conducting the review.

928 **16. Declaration of LLM usage**

929 Question: Does the paper describe the usage of LLMs if it is an important, original, or
 930 non-standard component of the core methods in this research? Note that if the LLM is used
 931 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
 932 scientific rigor, or originality of the research, declaration is not required.

933 Answer: [Yes]

934 Justification: LLMs were used for writing assistance, which does not require declaration per
 935 NeurIPS guidelines.

936 Guidelines:
 937 • The answer [N/A] means that the core method development in this research does not
 938 involve LLMs as any important, original, or non-standard components.
 939 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
 940 be described.